1. OpenClaw Deployment

========== [Note] ==========

This section serves as a comprehensive installation and deployment tutorial for OpenClaw. As it involves numerous steps, if you wish to start using OpenClaw quickly, please:

Follow the instructions in the first subsection of this chapter: [1. Directly Use the Pre-configured Environment (Recommended)].

==========

1. OpenClaw Deployment

- 1.1 OpenClaw Installation
  - 1.1.1 Installing OpenClaw
  - 1.1.3 Accessing the OpenClaw Control UI on a PC (Windows)
- 1.2 Configuring Models (Must Read)
  - 1.2.1 Why Configure Models?
  - 1.2.2 How to Configure Models
  - 1. OpenRouter (OpenClaw's Built-in Service Provider)
  - 2. Bailian (Example: Third-Party Service Provider API)

This chapter covers two main topics:

- OpenClaw Installation (Jetson Nano Docker Scenario)
- Model Configuration (OpenRouter / BaiLian / Ollama)

1.1 OpenClaw Installation

OpenClaw Official Website: https://openclaw.ai/

Open-Source Project Repository: https://github.com/openclaw/openclaw

1.1.1 Installing OpenClaw

The Jetson Nano B01 requires a Docker environment for operation. OpenClaw comes pre-configured within the official Yabo Intelligent system image; therefore, the content in this section is provided solely for your reference should you wish to perform the installation manually.

> The system version on the Jetson Nano B01 is relatively old and does not support Node.js versions higher than Node 22; consequently, installation via Docker is mandatory. ### 1.1.2 Installation using QuickStart

We will use the installation script officially provided by OpenClaw:

```
curl -fsSL https://openclaw.ai/install.sh | bash
```

```
root@jetson-yahboom:/# curl -fsSL https://openclaw.ai/install.sh | bash

  🦞 OpenClaw Installer
  Your messages, your servers, Meta's tears.

✓ Detected: linux

Install plan
OS: linux
Install method: npm
Requested version: latest

[1/3] Preparing environment
· Node.js not found, installing it now
· Installing Node.js via NodeSource
· Installing Linux build tools (make/g++/cmake/python3)
✓ Build tools installed
✓ Node.js v22 installed
· Active Node.js: v22.22.1 (/usr/bin/node)
· Active npm: 10.9.4 (/usr/bin/npm)

[2/3] Installing OpenClaw
✓ Git already installed
· Installing OpenClaw v2026.3.7
✓ OpenClaw npm package installed
✓ OpenClaw installed

[3/3] Finalizing setup

🦞 OpenClaw installed successfully (2026.3.7)!
Settled in. Time to automate your life whether you're ready or not.

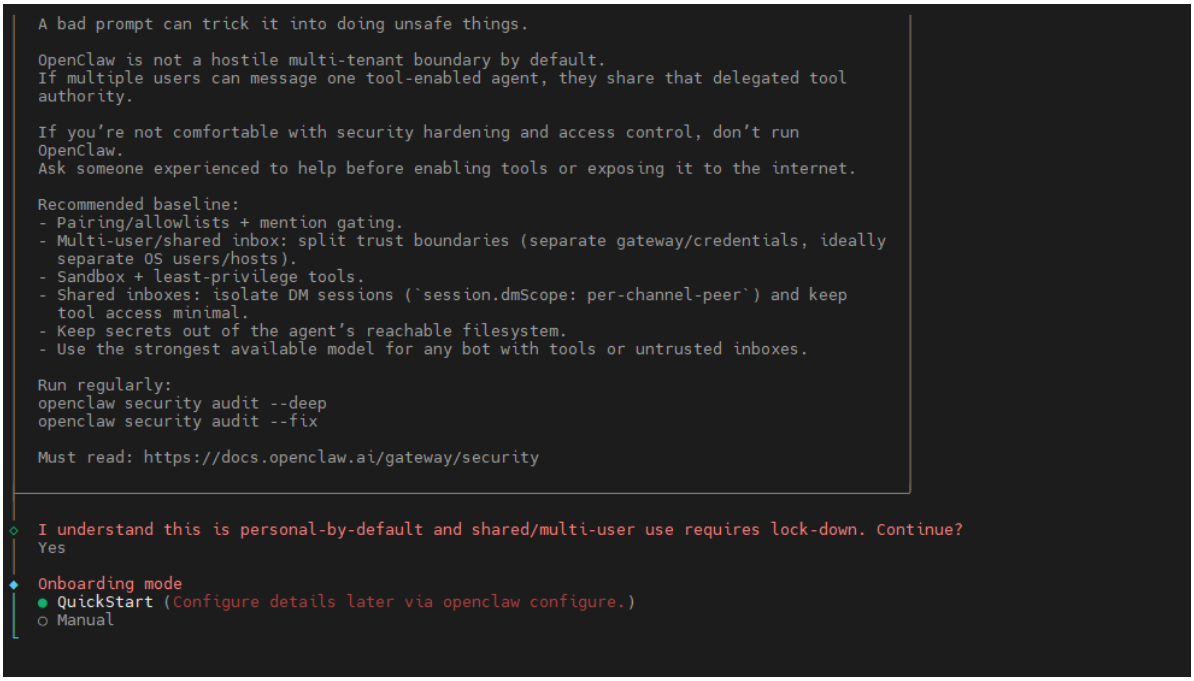
· Starting setup


🦞 OpenClaw 2026.3.7 (42a1394) — I run on caffeine, JSON5, and the audacity of "it worked on my machine."
```

During this process, the installation script will automatically detect any missing environment dependencies and assist us in installing them with a single click.

> During the installation process, use the arrow keys (Up, Down, Left, Right) on your keyboard to navigate options; press the Spacebar to select/deselect an item, and press Enter to confirm and proceed with configuration.

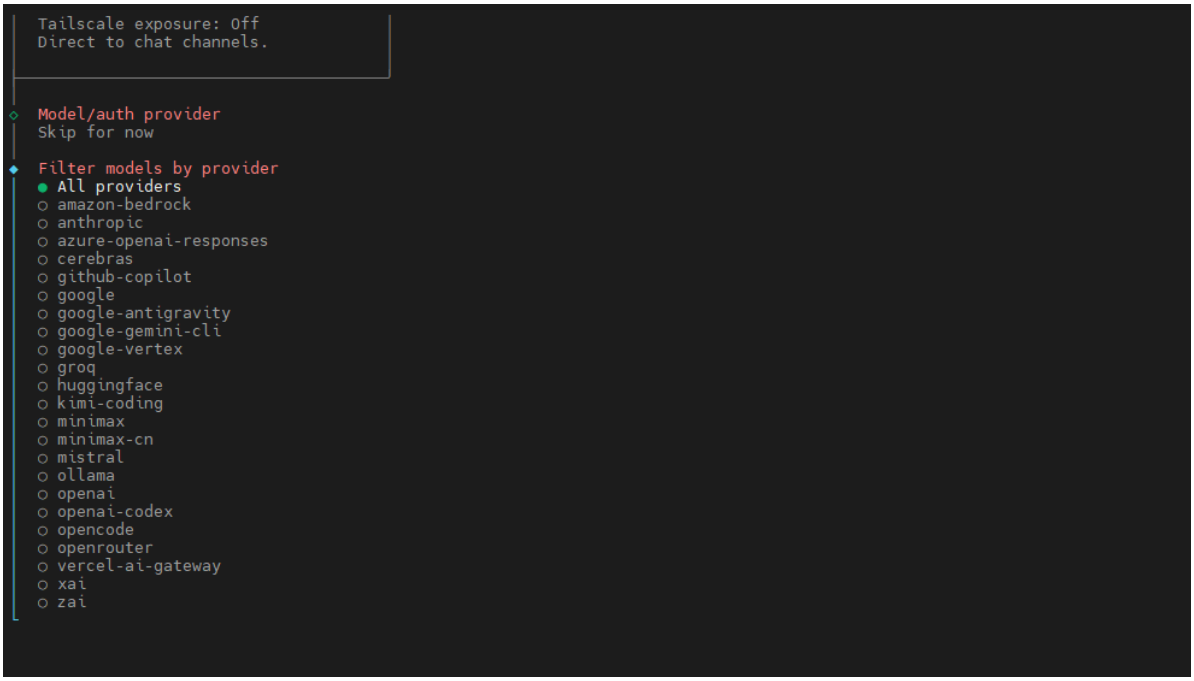




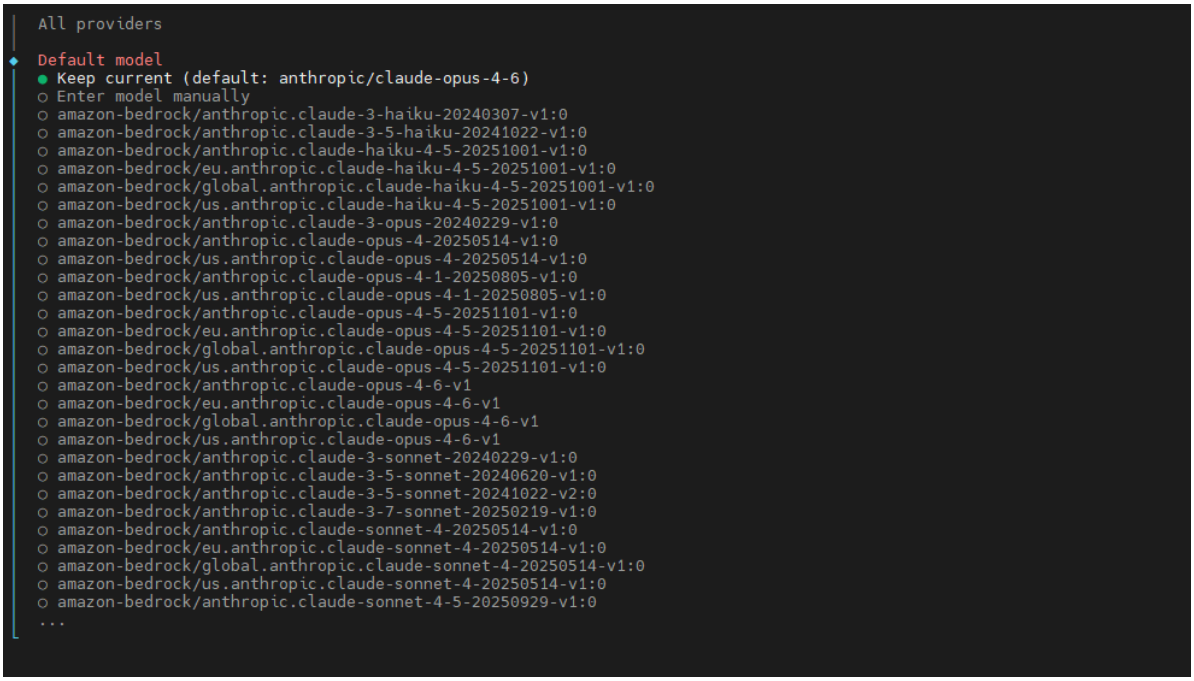
> Here, we will select the "QuickStart" option to rapidly complete the OpenClaw installation; we can defer the specific configuration details to a later stage.

If you have not yet applied for any API Keys, simply select "Skip" for these steps for now; you can configure them later.

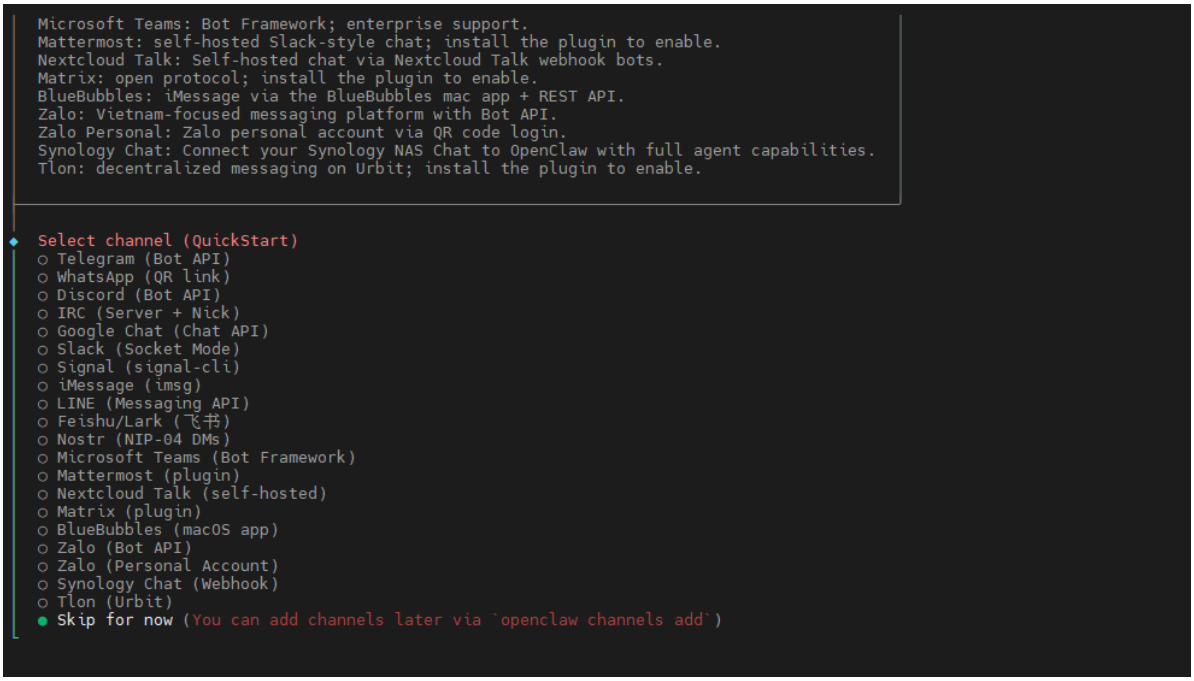




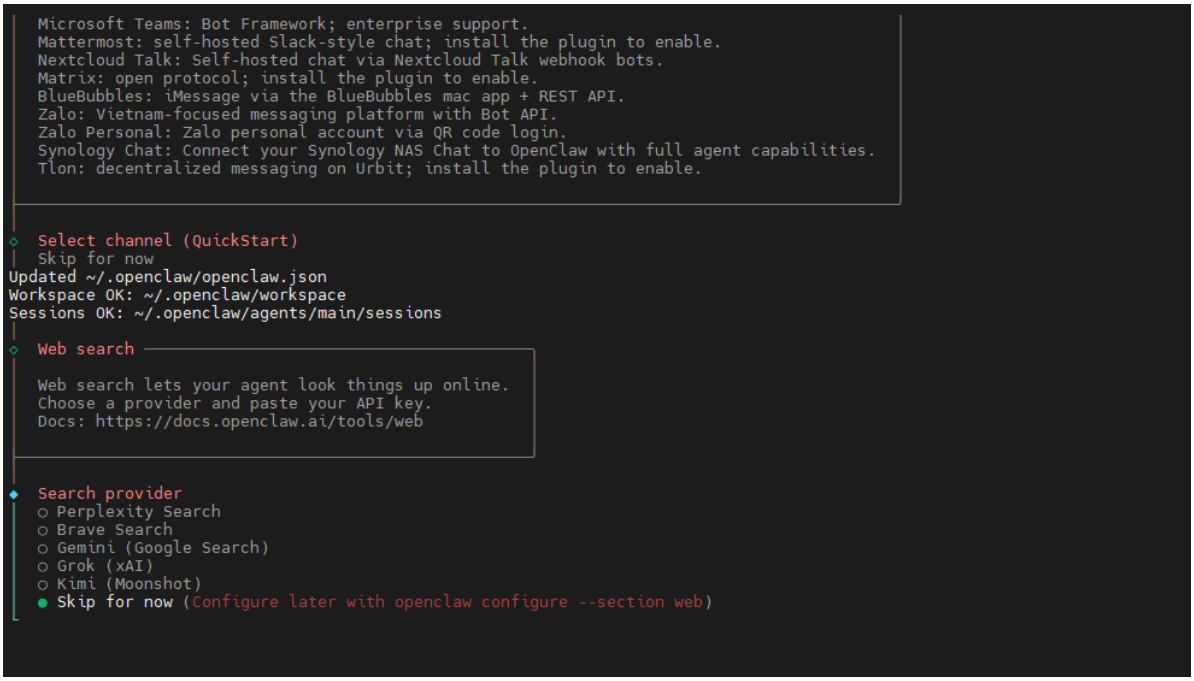
For the model selection, we will choose to keep the default settings (to be configured later).



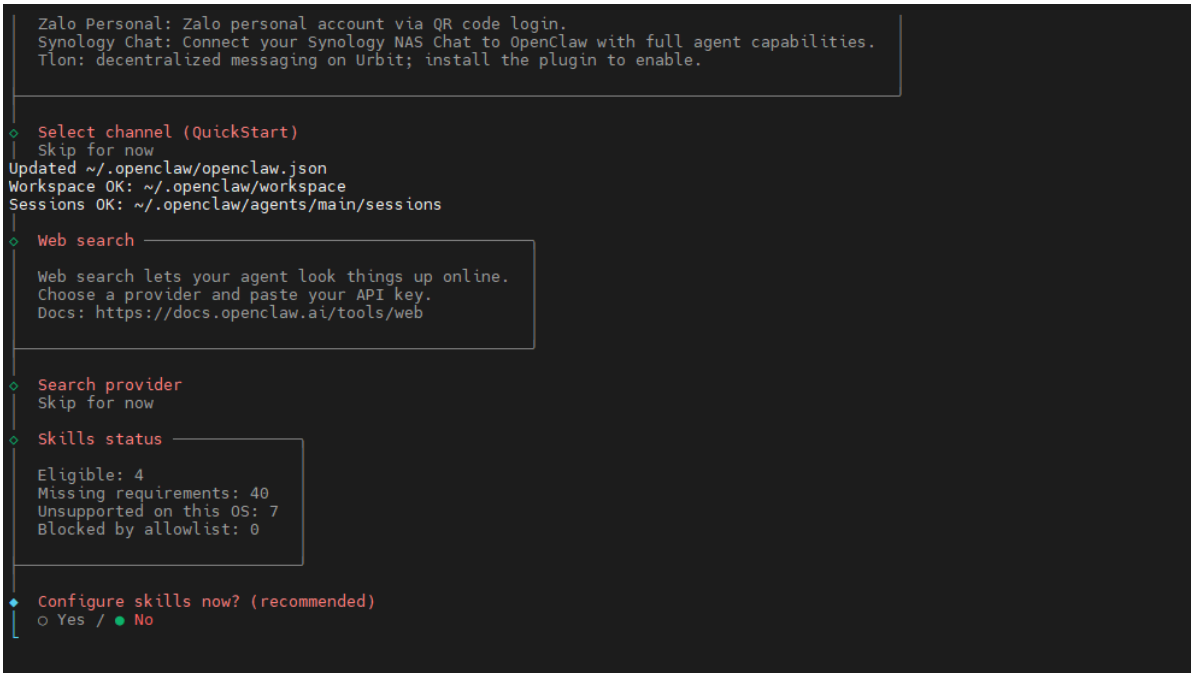
Regarding the "Channel" selection—which is primarily used for communication with third-party applications—we will also choose to skip it for the time being.



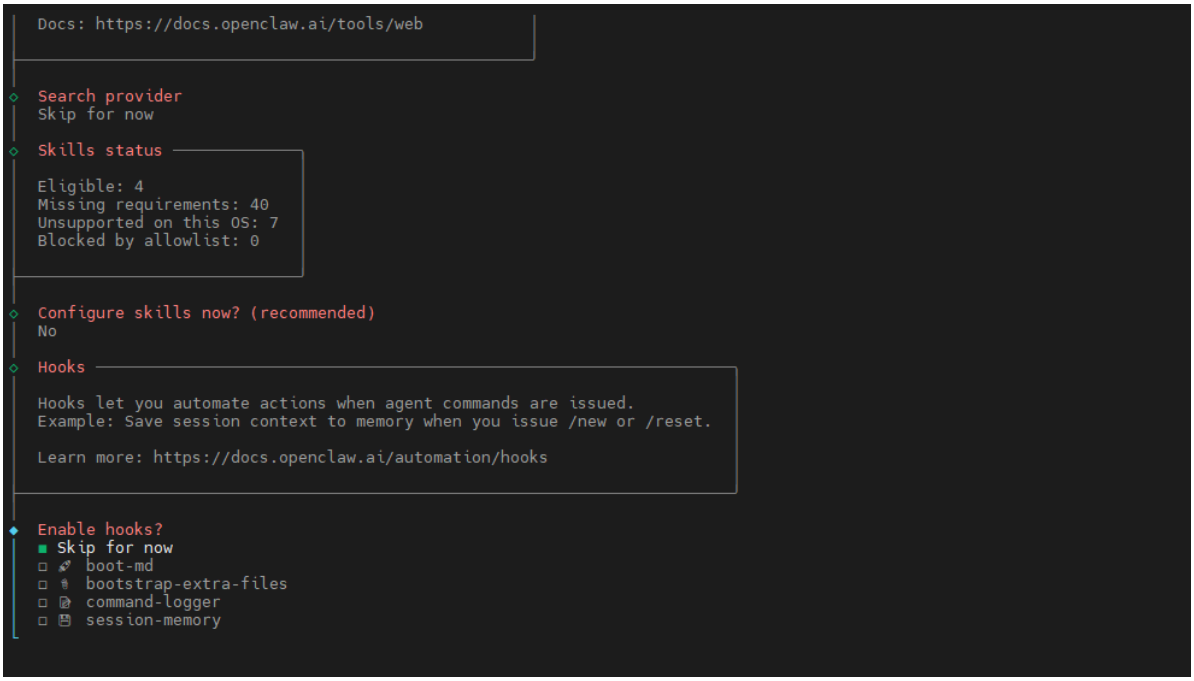
Search Provider refers to the API provider for search services; we choose to skip this step.



For the "Skills" configuration, we choose not to configure them for now (select "no").

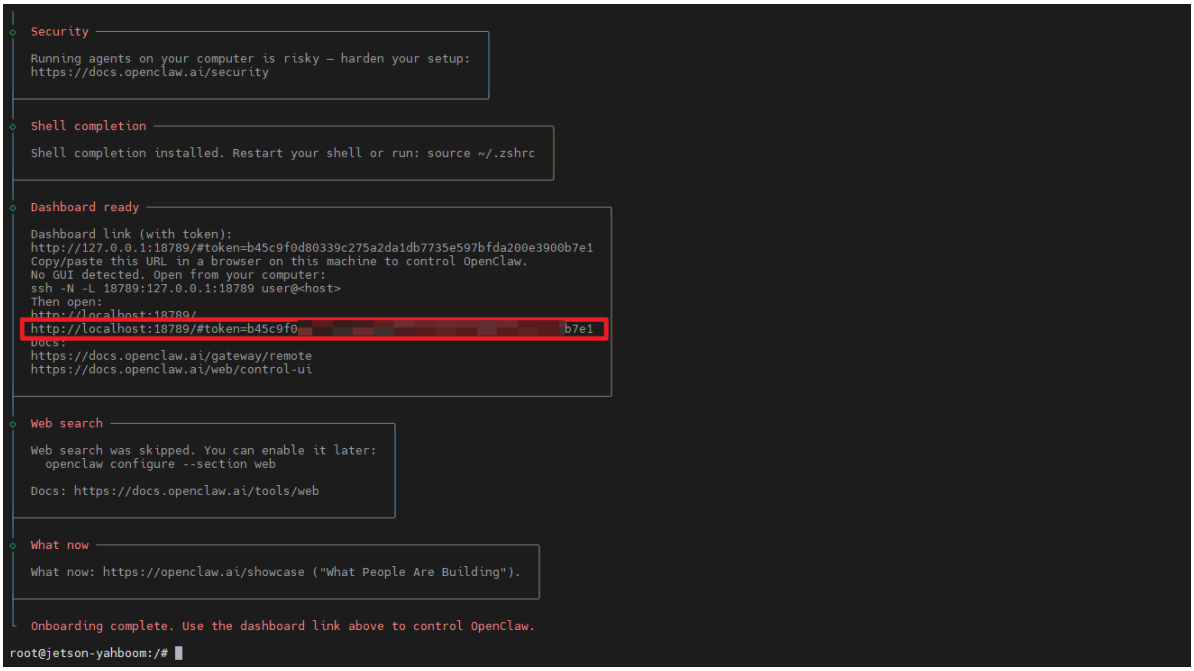


For the "Hooks" configuration, we press the Spacebar to select "Skip for now," and then press Enter to confirm.



Wait for the installation to complete.

Once the installation is finished, we copy this URL for future reference. (Note: In subsequent tutorials, you may observe that the token changes; this is because we later manually configured the Gateway. Therefore, it is not strictly necessary to record this specific URL here, as it will change later anyway.)



1.1.3 Accessing the OpenClaw Control UI on a PC (Windows)

> **Note on Common Issues:** Please use a recent version of the Edge or Chrome browser; other browsers may encounter a "4008" error.

After deploying OpenClaw version 3.8 within a Docker container on a Jetson Nano, attempting to access the OpenClaw Control UI (i.e., the web console) directly via a browser on the Jetson device itself will result in a "4008" access error. Therefore, you must access the interface using a browser on a separate computer.

> If no other computer is available, you can utilize OpenClaw's TUI tool, which operates via the command line within a terminal. For specific instructions, please refer to the "Using the TUI" section later in this document.

1. Modify the OpenClaw configuration file to allow other devices within your local network to access OpenClaw.

```
openclaw config
```

At this step, please make your selections as shown in the image below (use the Up/Down arrow keys to navigate, and press Enter to confirm).

> Note: The "Token" value may differ from what is shown in our image, but all other settings must match exactly. The critical setting here is the "Gateway bind mode" option; you must set this to "LAN" to enable other devices within your local area network to access your OpenClaw instance.



Finally, select "Continue" to confirm your choices, press Enter, and the configuration process is complete!



Once the configuration is complete, let's first start the OpenClaw Gateway. Open a new terminal window and enter the following command:

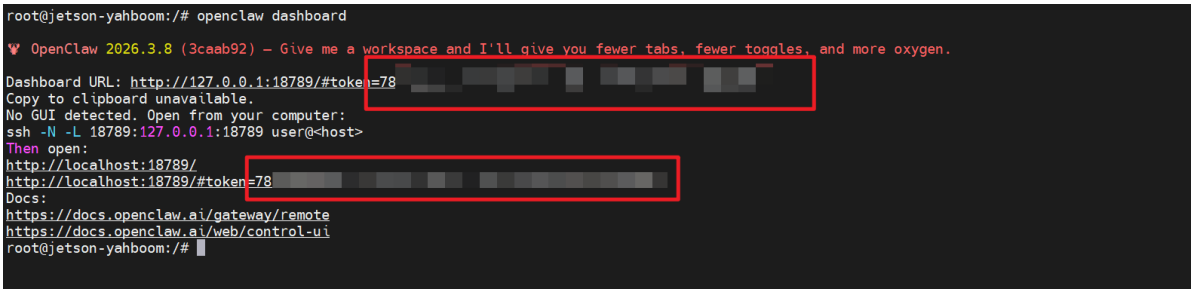
```
openclaw gateway
```

2. Obtain the Access Token

The Token referred to here is the one we generated during the configuration process. If you didn't write it down, you can use the following command:

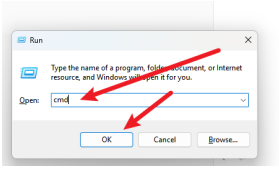
```
openclaw dashboard
```

This command will display the token for you once. Alternatively, you can locate the `~/.openclaw/openclaw.json` file; your token will be stored within this configuration file.



3. Configure the Connection Environment to the Jetson on a PC within the Same Local Network

Open a Command Prompt (Cmd) terminal on your PC. (Using Windows as an example: press `Win + R` to open the Run dialog, type `cmd`, and then press Enter.)

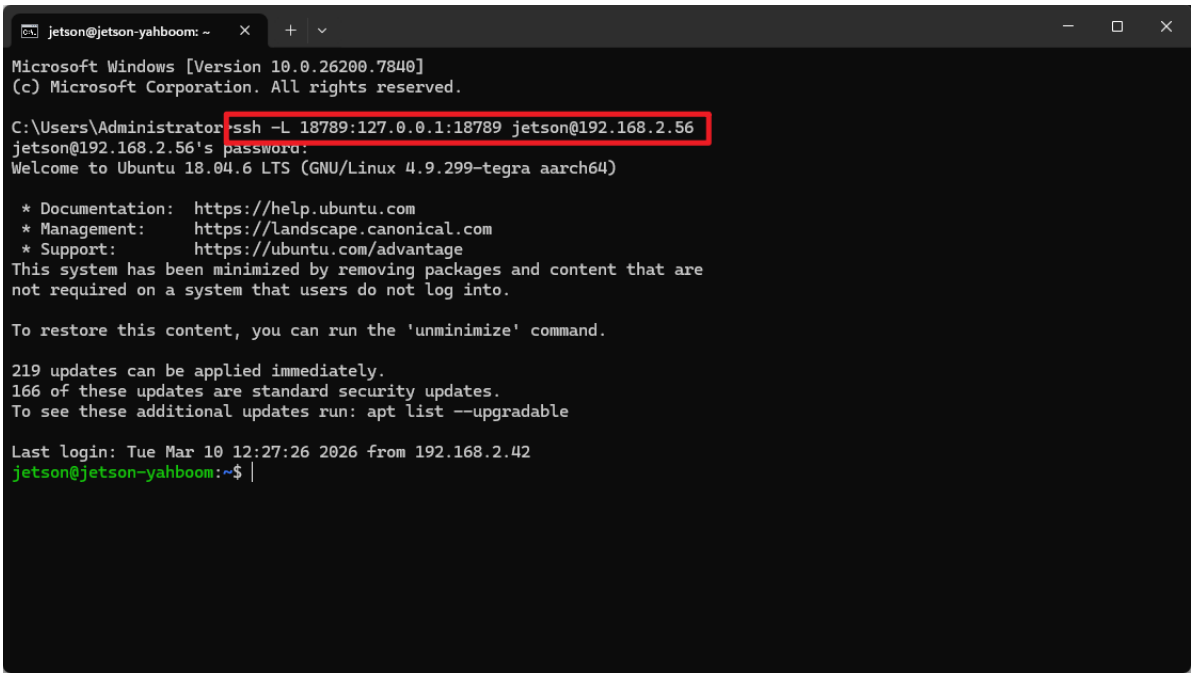


Enter the following command into the console:

```
ssh -L 18789:127.0.0.1:18789 jetson@<jetson-ip-address>
```

Note: You must replace `<jetson-ip-address>` with the actual IP address of your Jetson board.

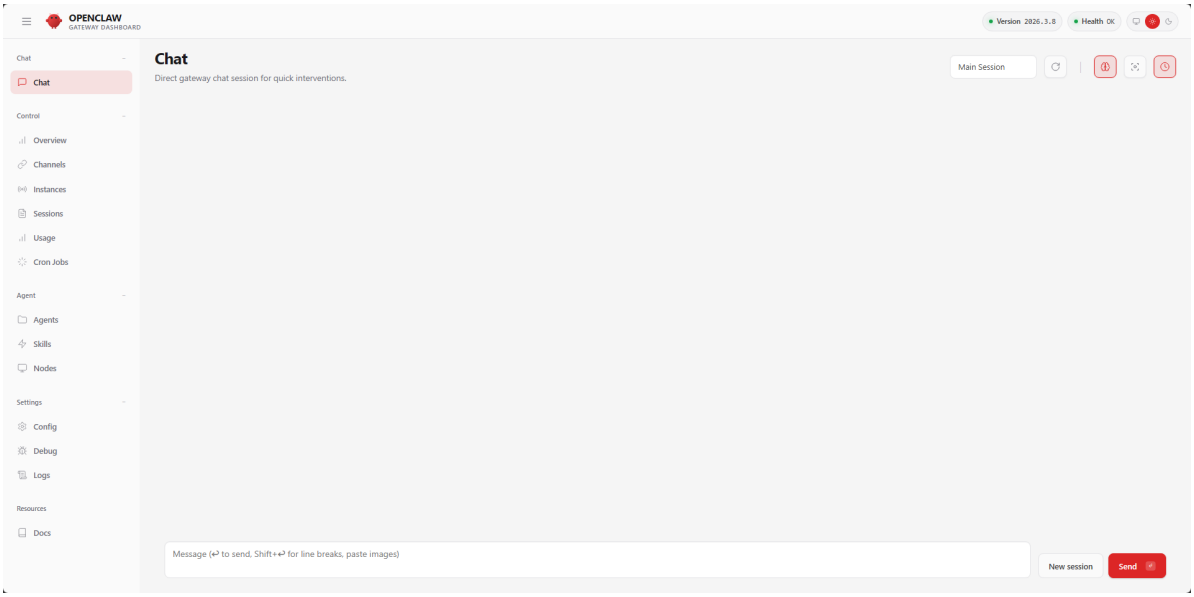
After pressing Enter, you will be prompted to enter the password (the default password is `yahboom`).



If this screen appears, it indicates that you have successfully connected to the Jetson.

4. Access via Web Browser

Open a web browser on your PC and enter the full address—consisting of the IP address combined with the token—into the address bar (this is the address we obtained using the `openclaw` command earlier). Once you see this interface at the address output by the dashboard (e.g., `https://127.0.0.1:18789/#token=xxxxxxxxxxxx`), it means the deployment was successful!



1.2 Configuring Models (Must Read)

1.2.1 Why Configure Models?

OpenClaw serves as an AI Agent gateway; on its own, it does not possess the "intelligent" capabilities of a large language model. We must configure a large model to endow OpenClaw with the ability to think and comprehend.

1.2.2 How to Configure Models

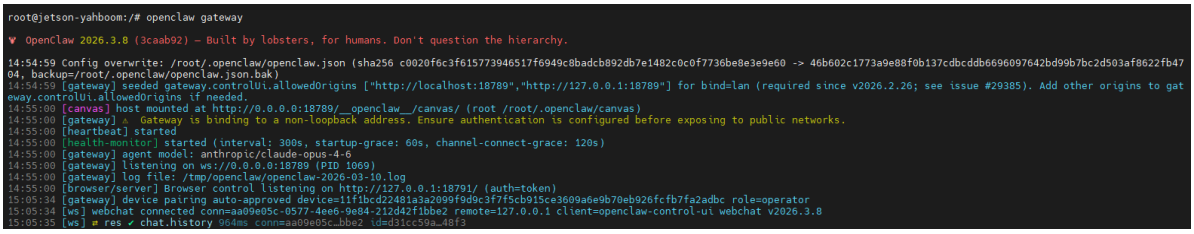
Here, we will use three representative models as examples:

1. OpenRouter (An OpenClaw-integrated provider, accessible both domestically and internationally)
2. Alibaba Bailian Large Model Platform (A third-party provider; requires manual configuration)
3. Ollama (A local large model)

Before you begin the configuration process, you must first start the OpenClaw Gateway. > If you did not exit the Gateway terminal after completing the deployment in the previous chapter, you may skip this step.

Enter the following command in your terminal:

```
openclaw gateway
```



1. OpenRouter (OpenClaw's Built-in Service Provider)

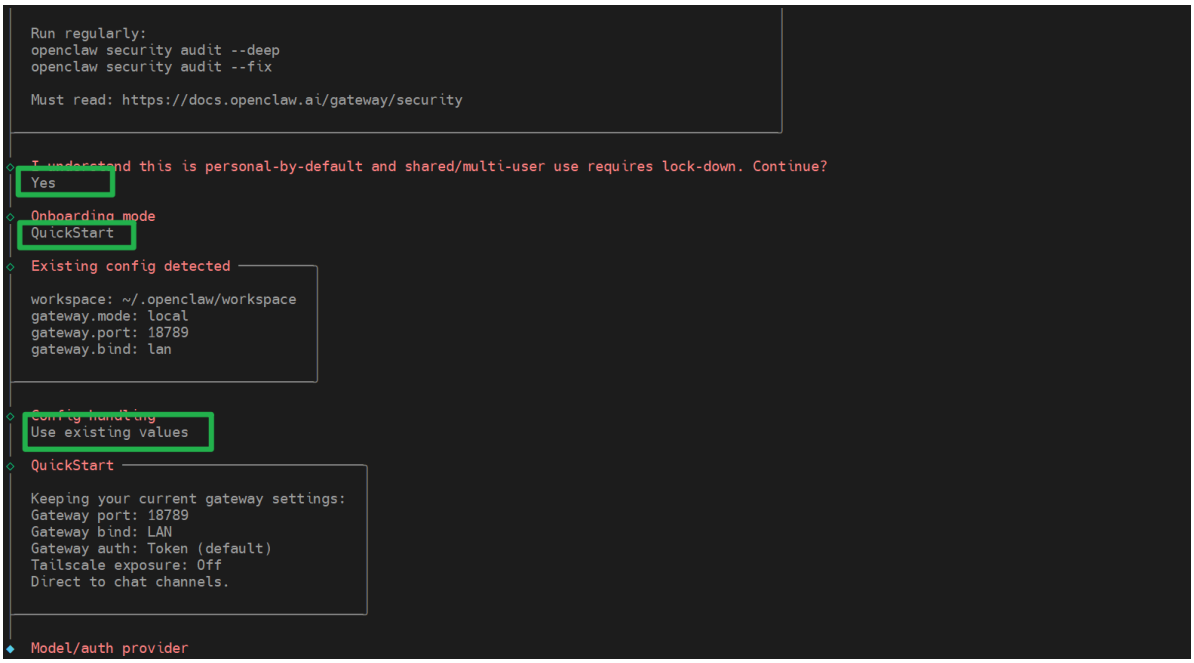
For this section, you must have already applied for an OpenRouter API Key by following the instructions in the previous tutorial (please refer to the earlier guide: [Online AI model course / 4. OpenRouter Large Model API Aggregation Platform]).

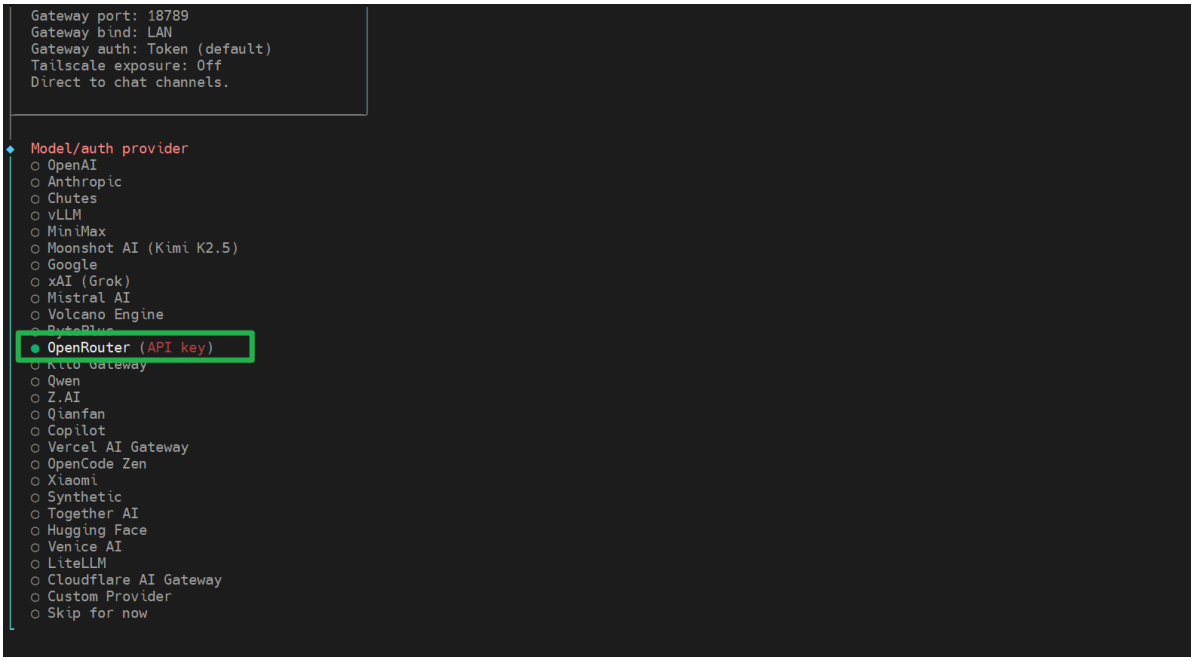
The format of an OpenRouter API Key typically looks like this: `sk-or-v1-b3exxxxxx4xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxcddf81d9`

We will once again use the initial "onboard" wizard to configure the settings.

```
openclaw onboard
```

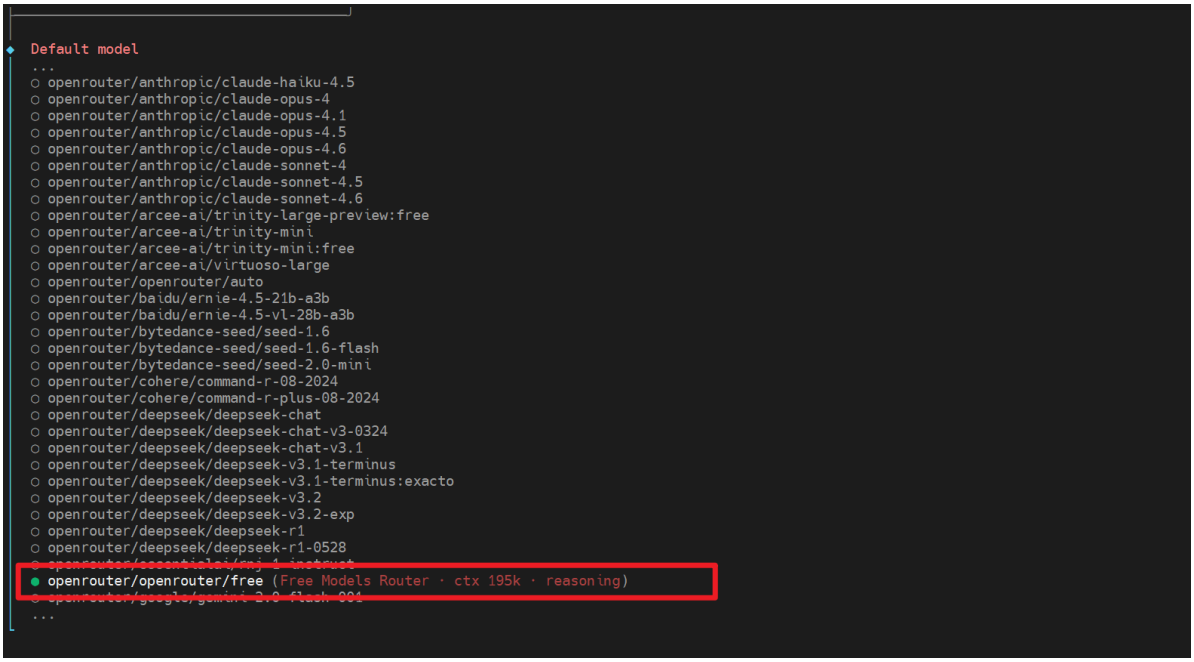
Please select the options exactly as shown in the images below.





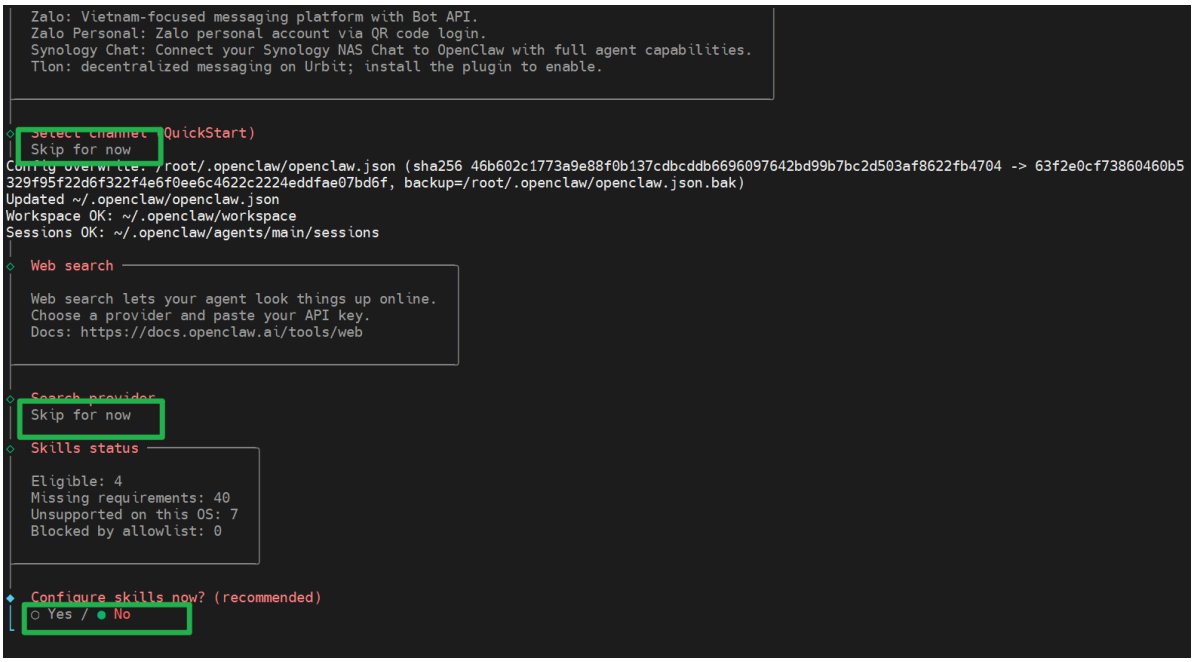
In the next step, we need to select a model. Due to OpenRouter's traffic control policies, we recommend maintaining an account balance of at least $11. If your balance falls below this threshold, OpenRouter may throttle your request speeds (please note that this policy is subject to change; always refer to the latest actual conditions). Here, we have the option to select a free

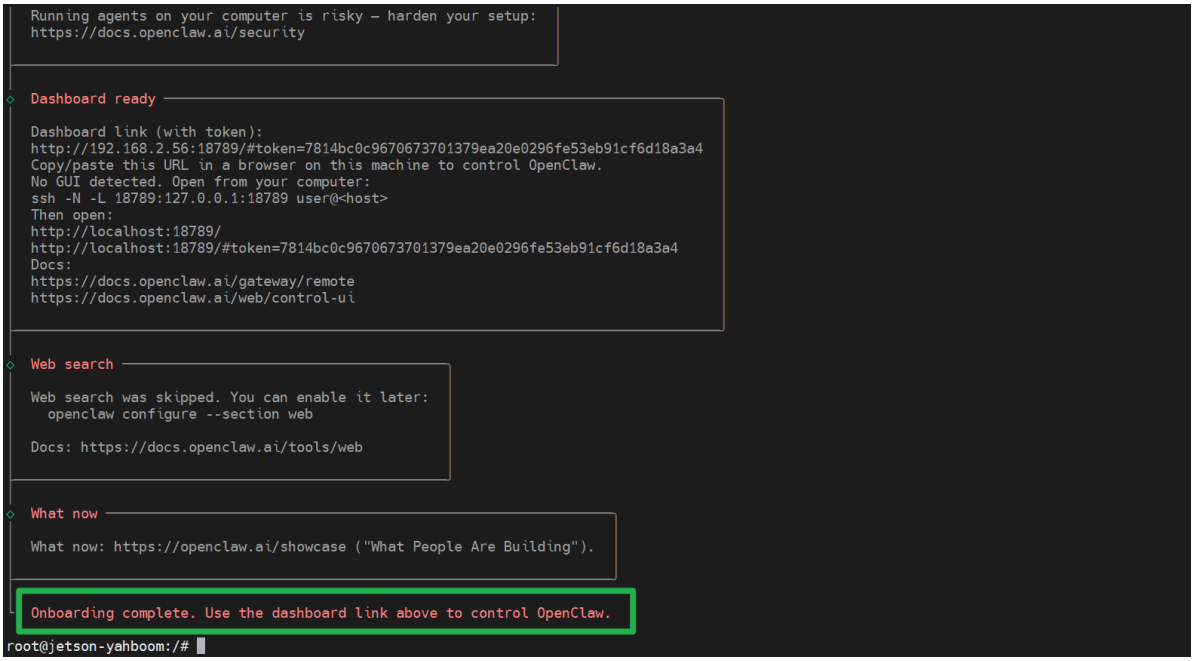
model (identified by model names ending in "free"), or to choose a dedicated free model provided by OpenRouter: `openrouter/free`. (This acts as a router that randomly selects a free model from those currently available on OpenRouter. It intelligently filters for models that support the specific capabilities required by your request.)



> At this stage, you may be prompted to enter an API Key. Simply paste your API Key into the field and press Enter to proceed to the next step.

Next, you will encounter various other configuration options. For now, we will skip these sections; select "Skip" for all of them.





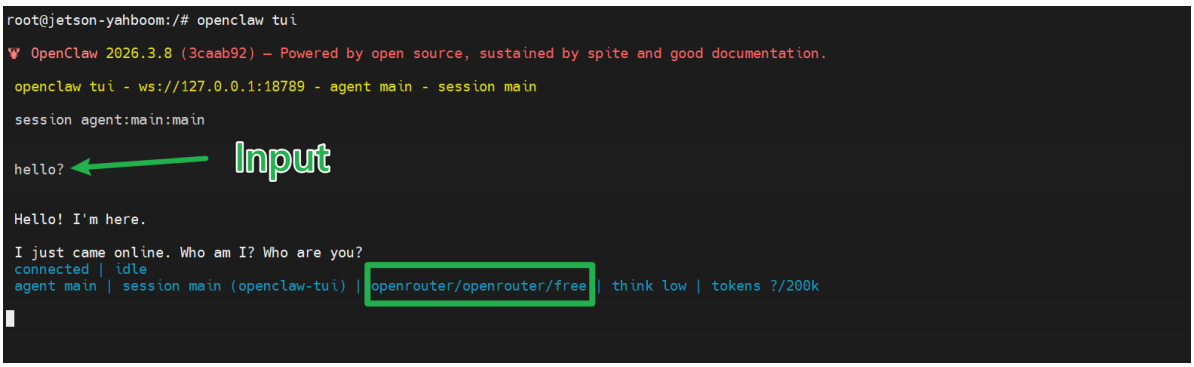
At this point, the configuration process is complete.

We can now use the TUI feature to perform a quick test.

Enter the following command:

```
openclaw tui
```

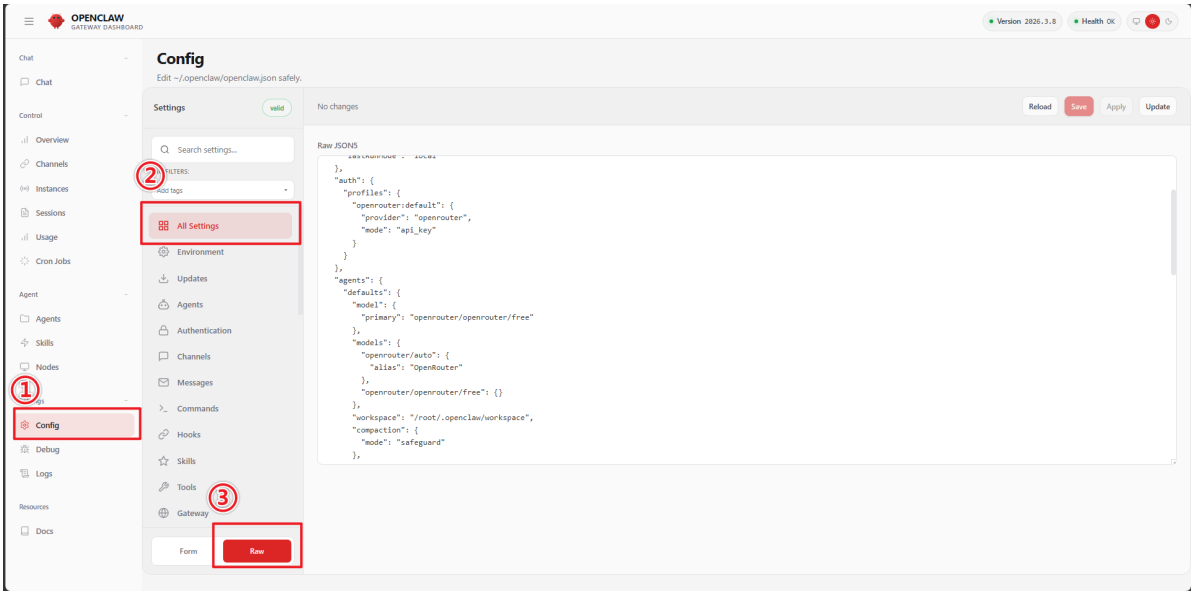
Once the command is entered, simply type a greeting to OpenClaw. (In the image below, the "Input" section displays the text you entered, while the green box highlights the model you selected.)



2. Bailian (Example: Third-Party Service Provider API)

Configuring a third-party API requires editing OpenClaw's configuration file (located at `~/.openclaw/openclaw.json`). However, I highly recommend performing these edits via the web interface (the OpenClaw Control UI) instead.

Open the OpenClaw configuration page in your web browser, then click the following three buttons in sequence:



The current contents of the configuration file are displayed below; pay particular attention to the `agents` section.

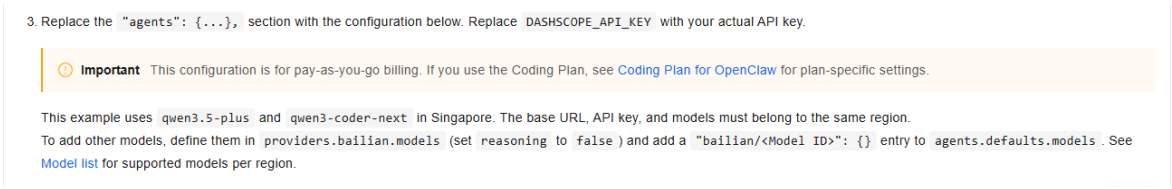

```
{
  "meta": {
    "lastTouchedVersion": "2026.3.8",
    "lastTouchedAt": "2026-03-10T07:56:18.292Z"
  },
  "wizard": {
    "lastRunAt": "2026-03-10T07:56:18.270Z",
    "lastRunVersion": "2026.3.8",
    "lastRunCommand": "onboard",
    "lastRunMode": "local"
  },
  "auth": {
    "profiles": {
      "openrouter:default": {
        "provider": "openrouter",
        "mode": "api_key"
      }
    }
  },
  "agents": {
    "defaults": {
      "model": {
        "primary": "openrouter/openrouter/free"
      },
      "models": {
        "openrouter/auto": {
          "alias": "OpenRouter"
        },
        "openrouter/openrouter/free": {}
      },
      "workspace": "/root/.openclaw/workspace",
      "compaction": {
        "mode": "safeguard"
      },
      "maxConcurrent": 4,
      "subagents": {
        "maxConcurrent": 8
```

```
      }
    }
  },
  "tools": {
    "profile": "coding"
  },
  "messages": {
    "ackReactionScope": "group-mentions"
  },
  "commands": {
    "native": "auto",
    "nativeSkills": "auto",
    "restart": true,
    "ownerDisplay": "raw"
  },
  "session": {
    "dmScope": "per-channel-peer"
  },
  "gateway": {
    "port": 18789,
    "mode": "local",
    "bind": "lan",
    "controlUi": {
      "allowedOrigins": [
        "http://localhost:18789",
        "http://127.0.0.1:18789"
      ]
    },
    "auth": {
      "mode": "token",
      "token": "__OPENCLAW_REDACTED__"
    },
    "tailscale": {
      "mode": "off",
      "resetOnExit": false
    },
    "nodes": {
      "denyCommands": [
        "camera.snap",
        "camera.clip",
        "screen.record",
        "contacts.add",
        "calendar.add",
        "reminders.add",
        "sms.send"
      ]
    }
  }
}
```

We need to add a "models" section (located at the top level of the JSON structure—meaning this newly added "models" object sits at the same hierarchical level as "agents").

Note: You need to replace `DASHSCOPE_API_KEY` with your actual Bailian API Key, and change `baseUrl` to the OpenAI Compatible Endpoint matching your account. (Refer to the earlier section for the application procedure.)

For the "models" section here, we have used `qwen3.5-plus` and `qwen3-coder-next` as examples. If you wish to support other models, please consult the model list available at: https://help.aliyun.com/zh/model-studio/models


3. Replace the `"agents": {...},` section with the configuration below. Replace `DASHSCOPE_API_KEY` with your actual API key.

Important This configuration is for pay-as-you-go billing. If you use the Coding Plan, see Coding Plan for OpenClaw for plan-specific settings.

This example uses `qwen3.5-plus` and `qwen3-coder-next` in Singapore. The base URL, API key, and models must belong to the same region.
To add other models, define them in `providers.bailian.models` (set `reasoning` to `false`) and add a `"bailian/<Model ID>": {}` entry to `agents.defaults.models`. See Model list for supported models per region.

```
"models": {
  "mode": "merge",
  "providers": {
    "bailian": {
      "baseUrl": "https://dashscope.aliyuncs.com/compatible-mode/v1",
      "apiKey": "DASHSCOPE_API_KEY",
      "api": "openai-completions",
      "models": [
        {
          "id": "qwen3.5-plus",
          "name": "qwen3.5-plus",
          "reasoning": false,
          "input": ["text", "image"],
          "contextWindow": 1000000,
          "maxTokens": 65536
        },
        {
          "id": "qwen3-coder-next",
          "name": "qwen3-coder-next",
          "reasoning": false,
          "input": ["text"],
          "contextWindow": 262144,
          "maxTokens": 65536
        }
      ]
    }
  }
},
```

After adding the "models," we also need to modify the "agents" section to set the BaiLian large language model as the default.

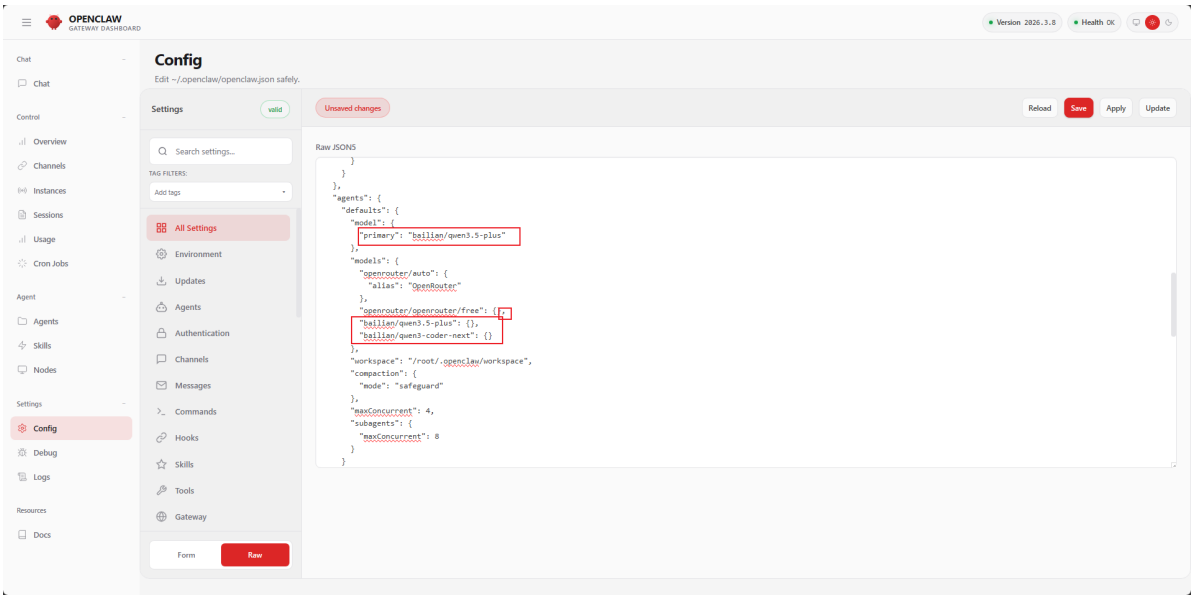
The "agents" configuration before modification is shown below (by default, it uses the OpenRouter service we configured in the previous step).

```
"agents": {
  "defaults": {
    "model": {
      "primary": "openrouter/openrouter/free"
    },
    "models": {
      "openrouter/auto": {
        "alias": "OpenRouter"
      },
      "openrouter/openrouter/free": {}
    },
    "workspace": "/root/.openclaw/workspace",
```

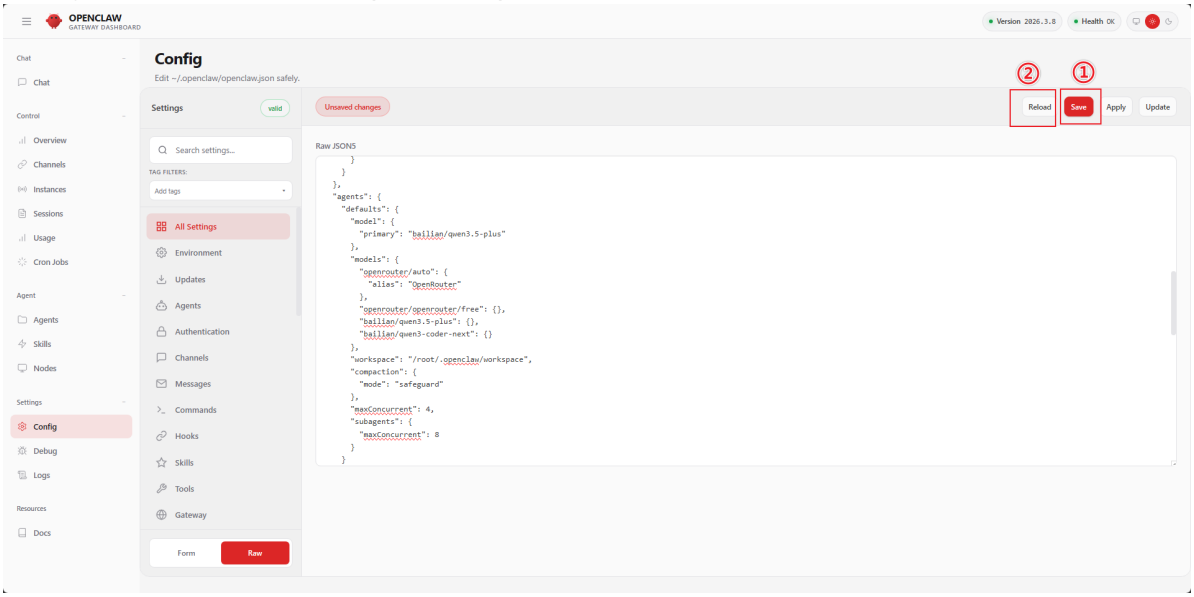
```
      "compaction": {
        "mode": "safeguard"
      },
      "maxConcurrent": 4,
      "subagents": {
        "maxConcurrent": 8
      }
    }
  },
```

The modified agents are as follows:

```
  "agents": {
    "defaults": {
      "model": {
        "primary": "bailian/qwen3.5-plus"
      },
      "models": {
        "openrouter/auto": {
          "alias": "OpenRouter"
        },
        "openrouter/openrouter/free": {},
        "bailian/qwen3.5-plus": {},
        "bailian/qwen3-coder-next": {}
      },
      "workspace": "/root/.openclaw/workspace",
      "compaction": {
        "mode": "safeguard"
      },
      "maxConcurrent": 4,
      "subagents": {
        "maxConcurrent": 8
      }
    }
  },
```



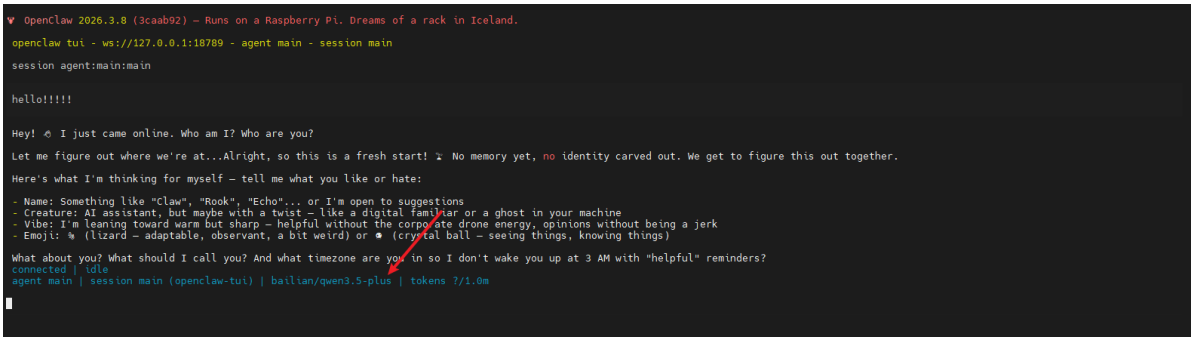
Once you have finished making the changes, remember to save and reload.



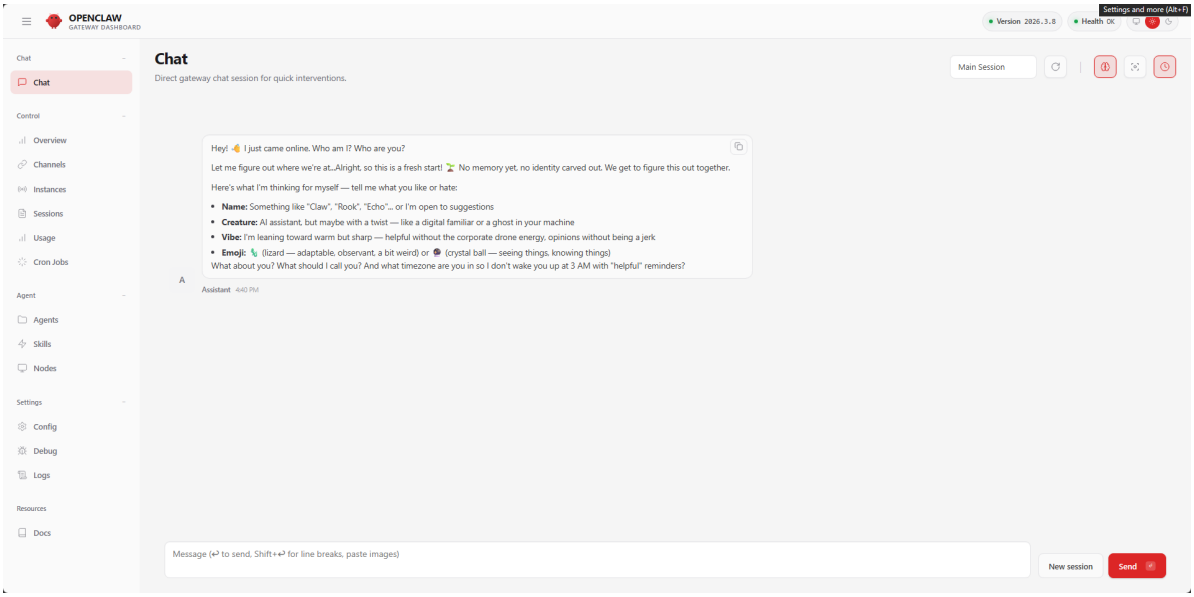
> The version released on March 8, 2026, supports hot-reloading of configuration files; after clicking "Save," there is no need to restart the OpenClaw service. Typically, simply restarting the OpenClaw TUI is sufficient. If restarting the TUI does not take effect, then try restarting the OpenClaw Gateway.

Initially, the status displayed here may show "unknown"; however, after waiting a short while, the current model will be correctly identified.





Similarly, in addition to the TUI interface, you can also engage in chat and dialogue within OpenClaw's Control UI (the web-based interface).



As you can observe, the conversation content displayed in the TUI is identical to that found in the Control UI. This is because OpenClaw's dialogue system utilizes a "Session" mechanism to isolate distinct conversations; in this instance, both the TUI and the Control UI are operating within the *same* session. You can try clicking the "New session" button located in the bottom-right corner to initiate a fresh conversation session.